# Color Pro Studio

**Documentación del Proyecto**

**Generador de paletas de colores profesionales**

## 1. 📸 Capturas del flujo principal

Esta sección presenta de forma visual el recorrido del usuario dentro de la aplicación, desde el ingreso inicial hasta la generación, personalización, uso y exportación de una paleta de colores.

### 1.1 Pantalla de inicio

Al ingresar a la aplicación, el usuario visualiza la pantalla principal del generador de paletas de colores.

En esta interfaz se encuentran los siguientes elementos:

- Título de la aplicación “Color Pro Studio”
- Selector de cantidad de colores (6, 8 o 9)
- Selector de formato de color (HEX o HSL)
- Botón principal “Generar Paleta”
- Interfaz visual donde se muestran las tarjetas de colores generadas

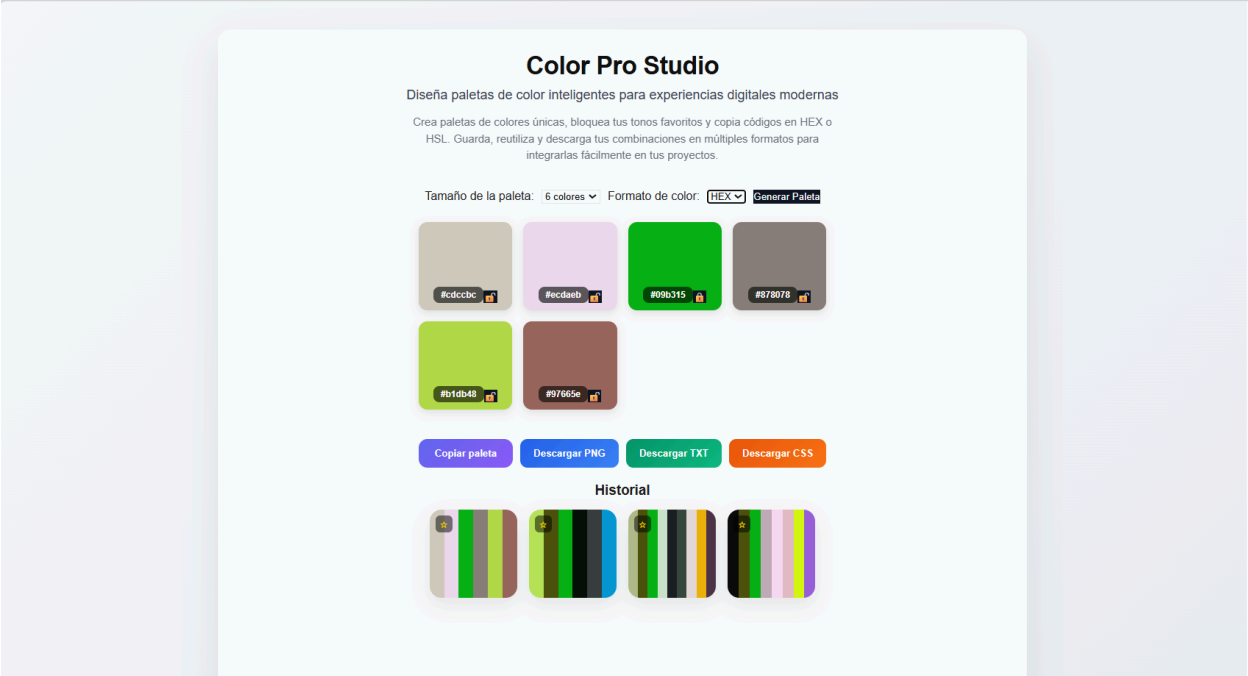
Debajo de la visualización de colores, se encuentran las opciones de interacción con la paleta:

- Botón “Copiar paleta”
- Botón de descarga en formato PNG
- Botón de descarga en formato TXT
- Botón de descarga en formato CSS

Finalmente, en la parte inferior de la interfaz se encuentra la sección de historial, donde se almacenan automáticamente las paletas generadas. Desde este historial el usuario puede:

- Volver a utilizar una paleta anterior
- Marcar una paleta como favorita
- Eliminar paletas del historial

Esta estructura permite una experiencia de usuario completa, facilitando la generación, reutilización y gestión de paletas de colores.




### 1.2 Configuración de la paleta

El usuario puede personalizar la generación de la paleta mediante controles ubicados en una misma fila en la parte superior de la interfaz. Desde allí es posible seleccionar la cantidad de colores (6, 8 o 9), elegir el formato de color (HEX o HSL) y ejecutar la generación de la paleta mediante el botón **"Generar Paleta"**.

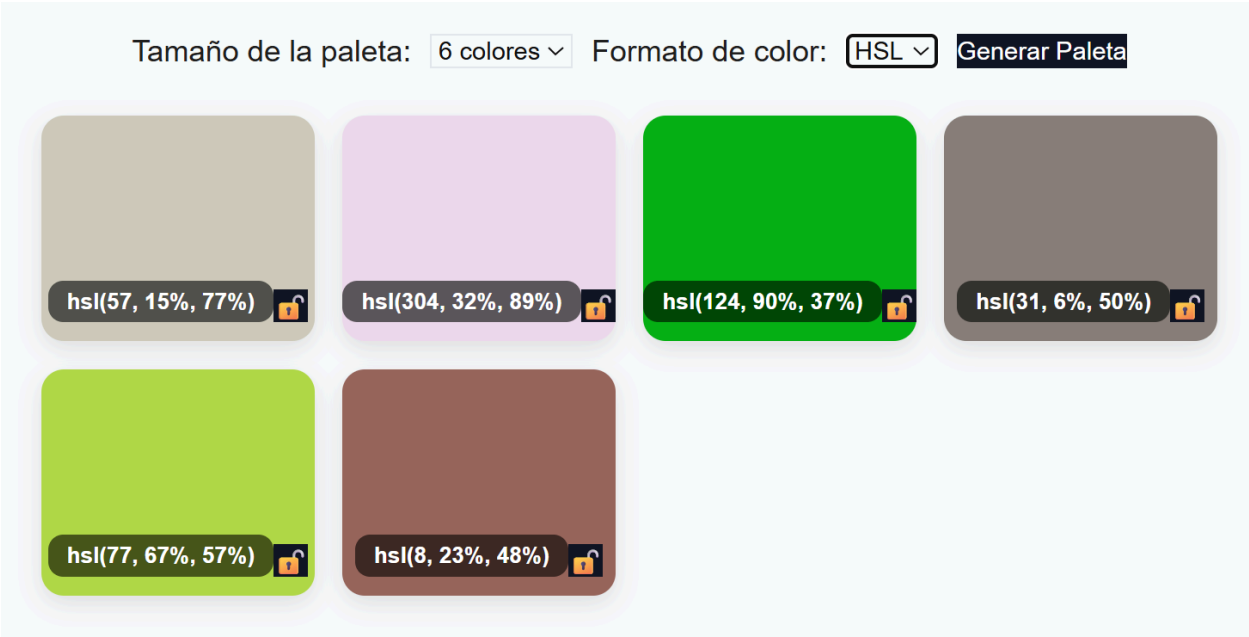
### 1.3 Generación de paleta

Al hacer clic en el botón **"Generar Paleta"**, la aplicación crea automáticamente una combinación de colores aleatorios basada en la configuración seleccionada. Cada color se muestra en bloques individuales junto con su respectivo valor de color.

El formato del valor mostrado (HEX o HSL) depende de la opción seleccionada en el control de formato ubicado en la parte superior de la interfaz. Al cambiar esta opción, los valores de los colores se actualizan automáticamente en toda la paleta. En la siguiente captura de pantalla vemos el formato HEX:

1.3.1 Cambio de formato de color

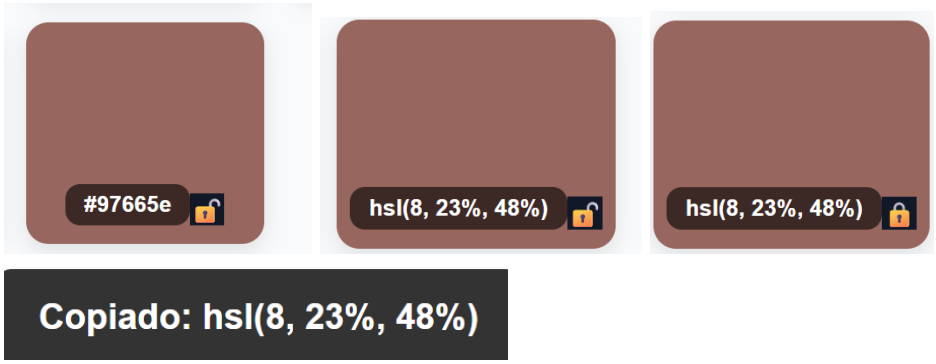
Al cambiar el formato de color a HSL desde el selector superior, los valores de cada color se actualizan automáticamente, manteniendo la misma paleta pero mostrando su representación en formato HSL.



### 1.4 Interacción con los colores

Cada bloque de color permite:

- Visualizar el código en formato **HEX o HSL**
- Bloquear colores para mantenerlos en futuras generaciones
- Copiar el color al portapapeles haciendo clic



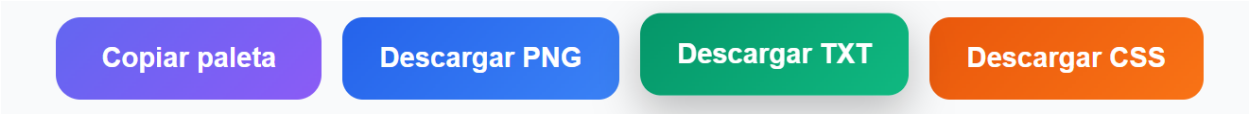

### 1.5 Exportación de la paleta

Una vez generada la paleta, el usuario dispone de múltiples opciones para exportar y reutilizar los colores generados.

Las opciones disponibles son:

- Copiar la paleta completa al portapapeles
- Descargar la paleta en formato PNG (imagen)
- Descargar la paleta en formato TXT (lista de colores)
- Descargar la paleta en formato CSS (variables listas para desarrollo web)

Estas funcionalidades permiten integrar fácilmente las paletas en proyectos de diseño, desarrollo web o herramientas externas.



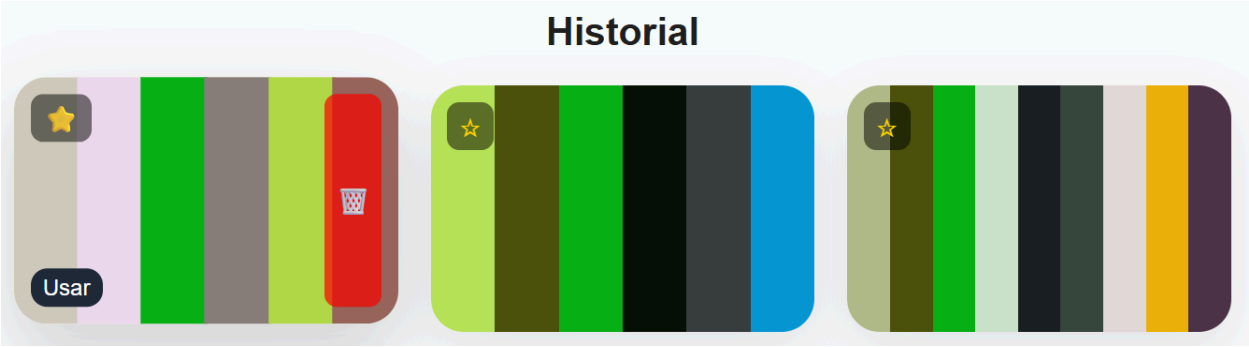

### 1.6 Historial de paletas

Las paletas generadas se almacenan automáticamente en el historial de la aplicación mediante el uso de almacenamiento local (localStorage).

Desde esta sección el usuario puede:

- Reutilizar una paleta anterior
- Marcar una paleta como favorita ⭐
- Eliminarla del historial 🗑️

Esto permite mantener un registro de combinaciones previamente generadas y facilita su reutilización en futuros proyectos.




**2. 🤖 Documentación del uso de IA**

Este proyecto se realizó con el apoyo de herramientas de inteligencia artificial, las cuales facilitaron el diseño, desarrollo y documentación del sistema.

**2.1 Herramientas de IA utilizadas**

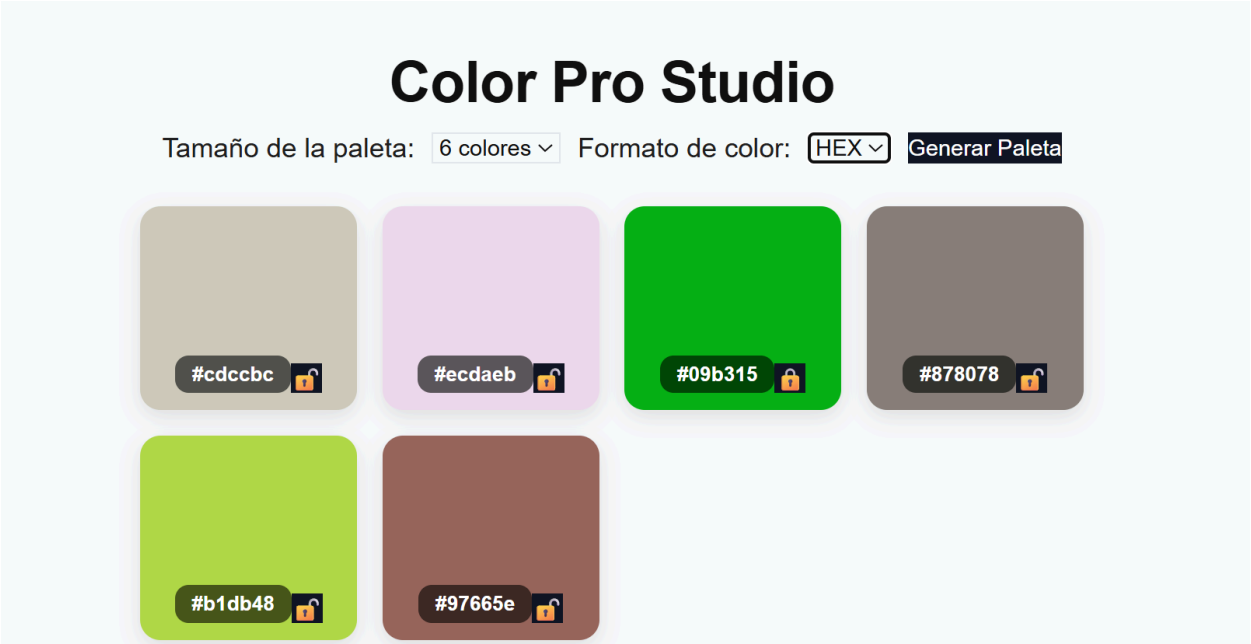
- ChatGPT (OpenAI)

  Asistencia en generación de código, lógica y documentación

**2.2 Prompts y resultados**

**Prompt 1: Estructura base del proyecto**

| **Prompt utilizado:** |
| --- |
| Necesito crear una aplicación web que genere paletas de colores aleatorias. Debe incluir un botón para generar la paleta, selección de tamaño (6, 8 o 9 colores), mostrar cada color en formato HEX y HSL, y permitir copiar y bloquear cada color. |
| **Resultado obtenido:** |
| La IA proporcionó la estructura base de la aplicación, incluyendo la lógica para generar colores, el diseño inicial de la interfaz y la interacción del usuario. El resultado fue una imagen de |




**Prompt 2: Conversión de HSL a HEX**

| **Prompt utilizado:** |
| --- |
| Trabajemos en una función en JavaScript que convierta un color en formato HSL a HEX. |
| **Resultado obtenido:** |
| La IA proporcionó una función hslToHex(hsl) que:<br>● Recibe un color en formato HSL como string<br>● Extrae los valores numéricos (H, S, L)<br>● Realiza la conversión matemática a RGB<br>● Convierte el resultado a formato HEX<br><br>Esta función permite generar colores en HSL y mostrarlos también en formato HEX dentro de la aplicación. |

```
cript.js > hslToHex
// HSL → HEX
// ======================
function hslToHex(hsl) {
    const result = hsl.match(/\d+/g);
    let h = result[0] / 360;
    let s = result[1] / 100;
    let l = result[2] / 100;

    let r, g, b;

    if (s === 0) {
        r = g = b = l;
    } else {
        const hue2rgb = (p, q, t) => {
            if (t < 0) t += 1;
            if (t > 1) t -= 1;
            if (t < 1 / 6) return p + (q - p) * 6 * t;
            if (t < 1 / 2) return q;
            if (t < 2 / 3) return p + (q - p) * (2 / 3 - t) * 6;
            return p;
        };

        const q = l < 0.5 ? l * (1 + s) : l + s - l * s;
        const p = 2 * l - q;

        r = hue2rgb(p, q, h + 1 / 3);
        g = hue2rgb(p, q, h);
        b = hue2rgb(p, q, h - 1 / 3);
    }

    const toHex = (x) => {
        const hex = Math.round(x * 255).toString(16);
        return hex.length === 1 ? "0" + hex : hex;
    };

    return `#${toHex(r)}${toHex(g)}${toHex(b)}`;
}
```

**Prompt 3: Exportación y descarga de paletas**

| **Prompt utilizado:** |
| --- |
| Trabajemos en funcionalidades en JavaScript que permitan exportar una paleta de colores generada, incluyendo opciones para copiar la paleta completa y descargarla en formatos PNG, TXT y CSS. |
| **Resultado obtenido:** |
| La IA proporcionó la lógica necesaria para implementar múltiples opciones de exportación de la paleta:<br>● Copiar la paleta completa al portapapeles<br>· Generar una imagen PNG<br>· Crear un archivo TXT con la lista de colores<br>· Generar un archivo CSS con variables listas para usar<br>Estas funcionalidades permiten al usuario reutilizar fácilmente las paletas en proyectos de diseño y desarrollo web, mejorando significativamente la utilidad práctica de la aplicación. |

ipt.js > downloadImageBtn.addEventListener("click") callback > currentPalette.forEach() callback

```
// ======================
// DESCARGAR COMO IMAGEN
// ======================
downloadImageBtn.addEventListener("click", () => {

    if (!currentPalette.length) {
        mostrarToast("No hay paleta para descargar");
        return;
    }

    const canvas = document.createElement("canvas");
    const ctx = canvas.getContext("2d");

    const width = 1000;
    const height = 300;

    canvas.width = width;
    canvas.height = height;

    const blockWidth = width / currentPalette.length;

    currentPalette.forEach((colorObj, i) => {
        ctx.fillStyle = colorObj.hex;
        ctx.fillRect(i * blockWidth, 0, blockWidth, height);

        ctx.fillStyle = "■#ffffff";
        ctx.font = "bold 16px Arial";
        ctx.textAlign = "center";

        const formato = colorFormat.value === "hsl" ? colorObj.hsl : colorObj.hex;

        ctx.fillText(
            formato,
            i * blockWidth + blockWidth / 2,
            height - 20
        );
```

```
    currentPalette.forEach((colorObj, i) => {
    });

    ctx.fillStyle = "□#000";
    ctx.font = "bold 20px Arial";
    ctx.fillText("Color Pro Studio", width / 2, 30);

    const link = document.createElement("a");
    link.download = "color-pro-studio.png";
    link.href = canvas.toDataURL();
    link.click();

    mostrarToast("Imagen descargada");
});

// ======================
// DESCARGAR COMO TXT
// ======================
downloadTxtBtn.addEventListener("click", () => {

    if (!currentPalette.length) {
        mostrarToast("No hay paleta para descargar");
        return;
    }

    const formato = colorFormat.value;

    const contenido = currentPalette.map(colorObj => {
        return formato === "hsl" ? colorObj.hsl : colorObj.hex;
    }).join("\n");

    const blob = new Blob([contenido], { type: "text/plain" });

    const link = document.createElement("a");
    link.href = URL.createObjectURL(blob);
```

```
    link.href = URL.createObjectURL(blob);
    link.download = "palette.txt";
    link.click();

    mostrarToast("Archivo TXT descargado");
});

// =======================
// DESCARGAR COMO CSS
// =======================
downloadCssBtn.addEventListener("click", () => {

    if (!currentPalette.length) {
        mostrarToast("No hay paleta para descargar");
        return;
    }

    let css = ":root {\n";

    currentPalette.forEach((colorObj, i) => {
        css += `  --color-${i + 1}: ${colorObj.hex};\n`;
    });

    css += "}";

    const blob = new Blob([css], { type: "text/css" });

    const link = document.createElement("a");
    link.href = URL.createObjectURL(blob);
    link.download = "palette.css";
    link.click();

    mostrarToast("Archivo CSS descargado");
});
```

**Prompt 4: Generación del README**

| **Prompt utilizado:** |
| --- |
| Necesito un README para mi proyecto de generador de paletas de colores que incluya descripción, funcionalidades, instrucciones de uso, ejecución local y despliegue en GitHub Pages. |
| **Resultado obtenido:** |
| La IA generó un README estructurado y completo que incluye:<br>· Descripción general del proyecto<br>· Funcionalidades principales de la aplicación<br>· Instrucciones de uso paso a paso<br>· Guía para ejecutar el proyecto en entorno local<br>· Instrucciones para desplegar la aplicación en GitHub Pages<br><br>El contenido generado fue posteriormente revisado, adaptado y mejorado para ajustarse a los requerimientos del proyecto y reflejar con precisión las funcionalidades implementadas en la aplicación. |

README.md

🎨 Color Pro Studio

Diseña paletas de color inteligentes para experiencias digitales.

Color Pro Studio es una aplicación web interactiva que permite generar paletas de colores aleatorias de forma rápida, visual e intuitiva. Está pensada para facilitar la creación de sistemas de color consistentes para diseño y desarrollo web.

Permite generar, bloquear, copiar y exportar paletas en múltiples formatos, mejorando el flujo de trabajo creativo.

🌐 Demo en vivo

👉 https://katherine-vasquez.github.io/proyectom1_katherinevasquez/

📷 Vista previa de la aplicación

<p *align*="center"> <img src="color-pro-studio-preview.png" width="700"/> </p>

🚀 Cómo usar la aplicación

Selecciona el tamaño de la paleta (6, 8 o 9 colores).
Elige el formato de color (HEX o HSL).
Haz clic en "Generar Paleta" para crear colores aleatorios.
Visualiza cada color con su código HEX y HSL.
Haz clic en un color para copiar su valor al portapapeles.
Usa el botón de "Copiar paleta" para copiar toda la paleta y pegarla en cualquier editor, proyecto o herramienta de diseño.
Bloquea los colores que quieras conservar usando 🔒.
Las paletas se guardan automáticamente en el historial del navegador (localStorage).
Desde el historial puedes reutilizar, marcar como favorita o eliminar paletas.
Exporta tus paletas en formatos PNG, TXT o CSS.

💻 Ejecución local

Si deseas ejecutar el proyecto en tu computadora:

Clona el repositorio:
git clone https://github.com/katherine-vasquez/proyectom1_katherinevasquez.git
Abre la carpeta del proyecto
Abre el archivo index.html en tu navegador

No se requieren dependencias adicionales.

✨ Funcionalidades principales

🎨 Generación de paletas de colores aleatorias
🔢 Selección de tamaño de paleta (6, 8 o 9 colores)
🎛️ Formato de color en HEX y HSL
📋 Visualización de cada color con su código HEX y HSL
🖱️ Copiar color individual al hacer clic
📋 Copiar paleta completa al portapapeles (para pegarla en cualquier herramienta o proyecto)
🔒 Bloqueo de colores individuales
💾 Guardado automático en localStorage
⭐ Sistema de favoritos
🗑️ Eliminación de paletas
📤 Exportación en PNG, TXT y CSS
⚡ Interfaz interactiva con microanimaciones

🧠 ¿Qué puede hacer el usuario?

El usuario puede generar combinaciones de colores automáticamente, ajustar la paleta bloqueando tonos específicos, copiar códigos de color para usarlos en otros proyectos, guardar sus combinaciones favoritas para reutilizarlas posteriormente y exportar sus paletas en diferentes formatos como imagen (PNG), texto (TXT) o variables CSS para desarrollo web.

📤 Exportación de paletas

El sistema permite exportar las paletas generadas de varias formas:

📋 Copiar paleta completa al portapapeles
💾 Descargar la paleta en diferentes formatos:
PNG (imagen de la paleta)
TXT (lista de códigos de color)
CSS (variables listas para usar en proyectos)

Esto facilita el uso de las combinaciones de color en otros proyectos de diseño, desarrollo web o UI/UX.

```md
README.md
🛠 Tech Stack
HTML5 → estructura semántica
CSS3 → diseño, layout y animaciones
JavaScript (Vanilla) → lógica e interacción
LocalStorage → persistencia de datos
Git / GitHub → control de versiones
GitHub Pages → despliegue

⚙ Funcionamiento técnico
Generación de colores mediante valores HSL aleatorios
Conversión automática de HSL a HEX
Renderizado dinámico del DOM para cada paleta
Gestión de estado con currentPalette
Persistencia de datos en localStorage
Sistema de historial limitado a las últimas 8 paletas
Manejo de eventos para interacción del usuario

🧠 Conocimientos aplicados
Manipulación del DOM
Manejo de eventos en JavaScript
Generación de valores aleatorios (HSL → HEX)
Renderizado dinámico de componentes
Persistencia con LocalStorage
Diseño UI/UX interactivo
Estructura de proyectos frontend

📁 Estructura del proyecto
proyectom1_katherinevasquez/
├── index.html
├── style.css
├── script.js
├── color-pro-studio-preview.png
└── README.md

🚀 Mejoras futuras (roadmap)
```

## 3. Entregable final

### 3.1 Repositorio en GitHub

- Repositorio: https://github.com/katherine-vasquez/proyectom1_katherinevasquez
- Rama principal: main
- Archivo README.md incluido en la raíz del proyecto
- Código fuente completo del proyecto (HTML, CSS y JavaScript)
- Historial de commits organizado que evidencia el proceso de desarrollo

---

### 3.2 Demo en producción

La aplicación se encuentra desplegada y accesible en:

https://katherine-vasquez.github.io/proyectom1_katherinevasquez/

### 3.3 Contenido del repositorio

El repositorio incluye:

1. README.md con documentación completa del proyecto
2. Código fuente completo (HTML, CSS y JavaScript)
3. Capturas de pantalla del flujo principal de la aplicación
4. Historial de commits organizado que evidencia el desarrollo
5. Link a la documentación del proyecto (Google Docs)

### 3.4 Checklist de entrega

✔ Repositorio GitHub público con el código fuente completo
✔ README.md completo con descripción e instrucciones de uso
✔ Aplicación funcional y operativa
✔ Aplicación desplegada y accesible mediante GitHub Pages
✔ Capturas del flujo principal de la aplicación incluidas
✔ Documentación del uso de IA con prompts y resultados
✔ Funcionalidades implementadas (generación, exportación e historial)
✔ Código organizado y estructurado (HTML, CSS, JavaScript)
✔ Link a la documentación del proyecto (Google Docs) incluido en el README

### 3.5 Enlace a documentación en Drive

**https://drive.google.com/drive/folders/1I6H32iINJm_sVhPRvo58nauA0QLmNNyF?usp=drive_link**

**Autora**

**Katherine Vasquez**